CLOSECUT LIBRARY / VOLUME 05

Quality, Release & Operations

The Public Validation and Delivery Specification

PUBLIC READING EDITION

Version 1.0 · July 2026

Contents

Quality Philosophy

Purpose

Establish the quality contract for quality philosophy so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Quality Principles

Purpose

Establish the quality contract for quality principles so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Product Quality Bar

Purpose

Establish the quality contract for product quality bar so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Risk-Based Testing

Purpose

Establish the quality contract for risk-based testing so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Evidence and Traceability

Purpose

Establish the quality contract for evidence and traceability so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Definition of Ready

Purpose

Establish the quality contract for definition of ready so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Definition of Done

Purpose

Establish the quality contract for definition of done so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Release Confidence Model

Purpose

Establish the quality contract for release confidence model so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Test Strategy Overview

Purpose

Establish the quality contract for test strategy overview so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Test Pyramid

Purpose

Establish the quality contract for test pyramid so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Unit Testing

Purpose

Establish the quality contract for unit testing so release decisions are based on observable evidence and user risk.

Current state

The repository contains 48 unit-level cases across focused domain and sync components.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Expand coverage to service, repository, session, profile, sharing and deletion behavior.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Integration Testing

Purpose

Establish the quality contract for integration testing so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

UI Testing

Purpose

Establish the quality contract for ui testing so release decisions are based on observable evidence and user risk.

Current state

One launch UI smoke test was identified.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Build stable launch arguments, seeded stores and page objects for critical flows.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Manual and Exploratory Testing

Purpose

Establish the quality contract for manual testing so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Regression and Smoke Testing

Purpose

Establish the quality contract for regression testing so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Performance and Reliability Testing

Purpose

Establish the quality contract for performance testing so release decisions are based on observable evidence and user risk.

Current state

No dedicated benchmark target or retained performance baseline was found.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Create repeatable budgets and baseline measurements on representative devices.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Privacy Verification

Purpose

Establish the quality contract for privacy verification so release decisions are based on observable evidence and user risk.

Current state

Rules and privacy documentation exist; executable rules verification and end-to-end privacy evidence were not found in the test targets.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Add emulator-based allow/deny tests and release evidence for every data boundary.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Accessibility Testing

Purpose

Establish the quality contract for accessibility testing so release decisions are based on observable evidence and user risk.

Current state

An accessibility audit document exists, but systematic automated and device execution evidence was not found.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Run and record the accessibility matrix on release-candidate builds.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Device and OS Coverage

Purpose

Establish the quality contract for device and os coverage so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Offline Quality Philosophy

Purpose

Establish the quality contract for offline quality philosophy so release decisions are based on observable evidence and user risk.

Current state

Focused unit coverage exists for pending-action queue behavior and entry conflict policy; remote integration evidence is incomplete.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Add emulator-backed, termination, reconnection, multi-device and partial-failure scenarios.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Pending Actions, Retry, and Conflict Testing

Purpose

Establish the quality contract for pending action queue testing so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Data Loss and Duplicate Prevention

Purpose

Establish the quality contract for data loss prevention so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Recovery from Partial Failure

Purpose

Establish the quality contract for recovery from partial failure so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

TestFlight Strategy

Purpose

Establish the quality contract for testflight strategy so release decisions are based on observable evidence and user risk.

Current state

Release backlog and known limitations exist; no complete, repeatable beta evidence package was found.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define cohorts, build assignment, feedback taxonomy, exit criteria and incident handling.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Beta Exit Criteria

Purpose

Establish the quality contract for beta exit criteria so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

App Store Release Philosophy

Purpose

Establish the quality contract for app store release philosophy so release decisions are based on observable evidence and user risk.

Current state

App Store readiness cannot be established from the repository alone.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Verify App Store Connect fields against the release candidate and retain dated screenshots/evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Release Candidate and Gates

Purpose

Establish the quality contract for release candidate so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Post-Release Validation and Rollback

Purpose

Establish the quality contract for post-release validation so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Operational Philosophy

Purpose

Establish the quality contract for operational philosophy so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Observability and Monitoring

Purpose

Establish the quality contract for observability so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Support and Feedback

Purpose

Establish the quality contract for support philosophy so release decisions are based on observable evidence and user risk.

Current state

Operational intent is documented in prior books, but production telemetry and runbook execution evidence are not yet demonstrated.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Configure ownership, alerts, thresholds, drills and privacy-safe diagnostics before production.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Cross-Platform Quality

Purpose

Establish the quality contract for cross-platform quality principles so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Quality Governance

Purpose

Establish the quality contract for quality governance so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

QDR-001, QDR-008, QDR-010, QDR-014.

Quality Decision Records

Purpose

Establish the quality contract for quality decision records so release decisions are based on observable evidence and user risk.

Current state

No direct automated evidence was identified for this subject.

Quality risk

This area must be assessed by user harm, reversibility, privacy boundary, data durability, platform dependency and frequency of change. Unknown behavior is treated as risk, not as a pass.

Required coverage

Use the lowest-cost test level that can prove the behavior, then add integration or UI coverage where framework boundaries, persistence, permissions, accessibility, remote authorization or user-visible recovery are involved. Use deterministic fixtures; capture build, device, OS, environment and result.

Missing coverage

Define measurable coverage and retain execution evidence.

Acceptance criteria

The defined happy path and negative paths pass; failures are visible and recoverable; no unauthorized data is exposed; core controls are accessible; offline behavior preserves intent; and evidence is linked to the release candidate.

Release gate

A failure blocks release when it can cause data loss, privacy breach, inaccessible core flow, account lockout or unrecoverable user state.

Evidence required

Retain the test run, build/version, commit, environment, device/OS, fixture or account, result, defects and approver. Manual evidence expires when affected code, configuration or policy changes.

Ownership

Engineering owns automated correctness; the Release Owner owns execution evidence; Product owns acceptance; the Accessibility and Privacy reviewers own their respective gates.

Near-term improvement

Close the highest-risk evidence gap before increasing breadth. Prefer a small deterministic suite over broad flaky automation.

Long-term maturity

Automate repeatable checks, trend outcomes, and keep platform-specific exceptions explicit while preserving shared acceptance criteria.

Related decisions

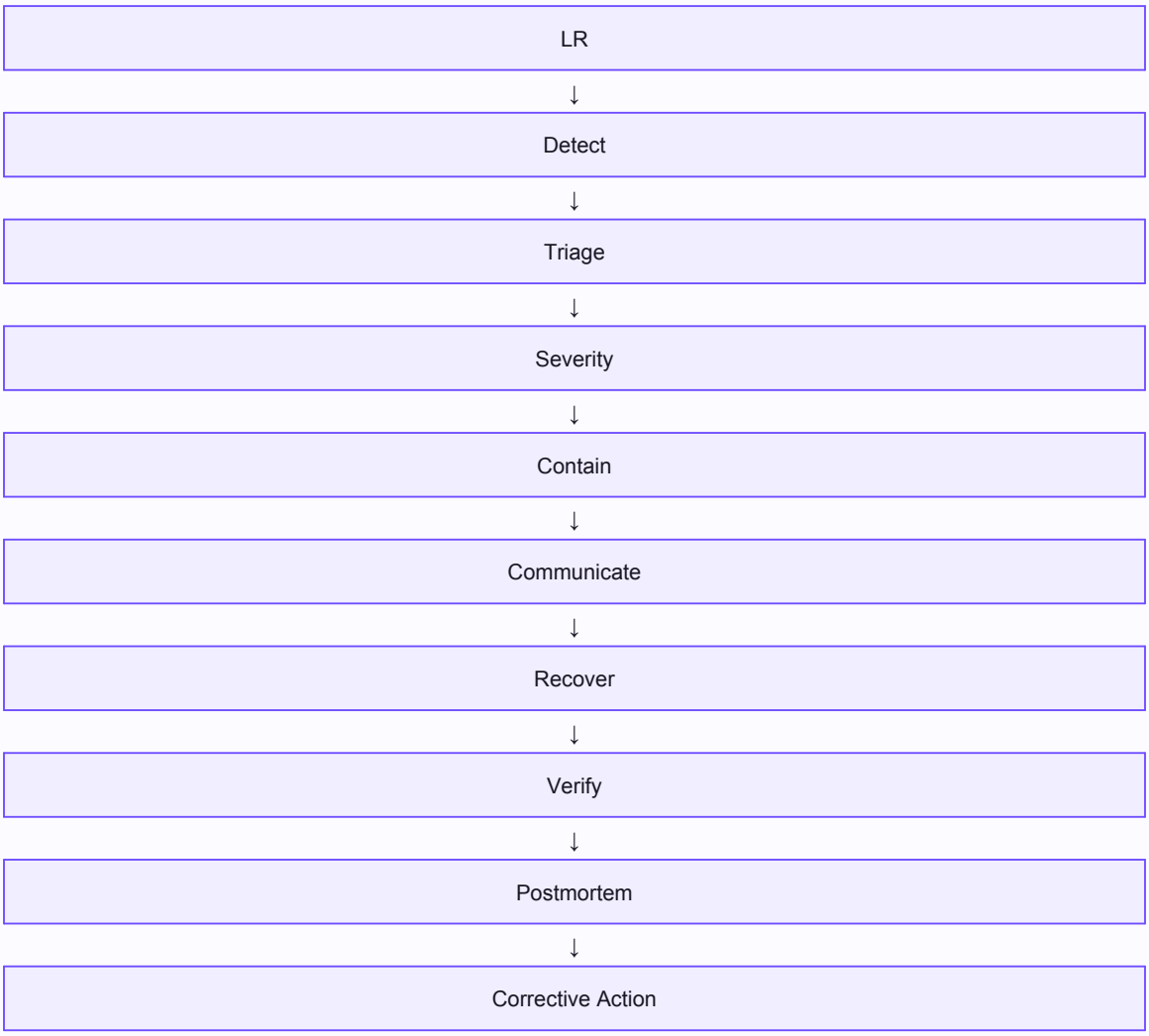
QDR-001, QDR-008, QDR-010, QDR-014.

Cross Platform Parity

Public quality workflow diagram

Incident Response

Public quality workflow diagram

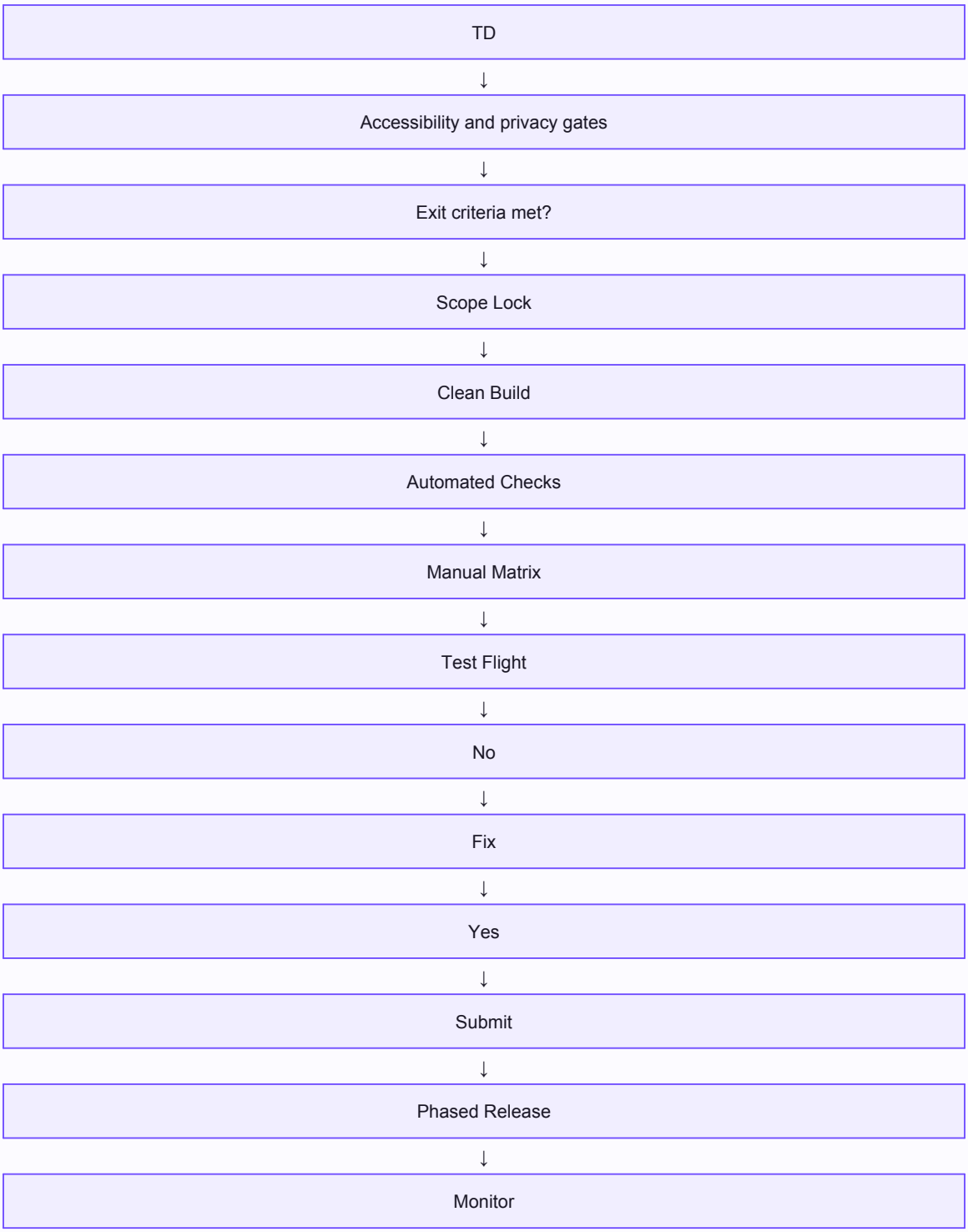


Quality Lifecycle

Public quality workflow diagram

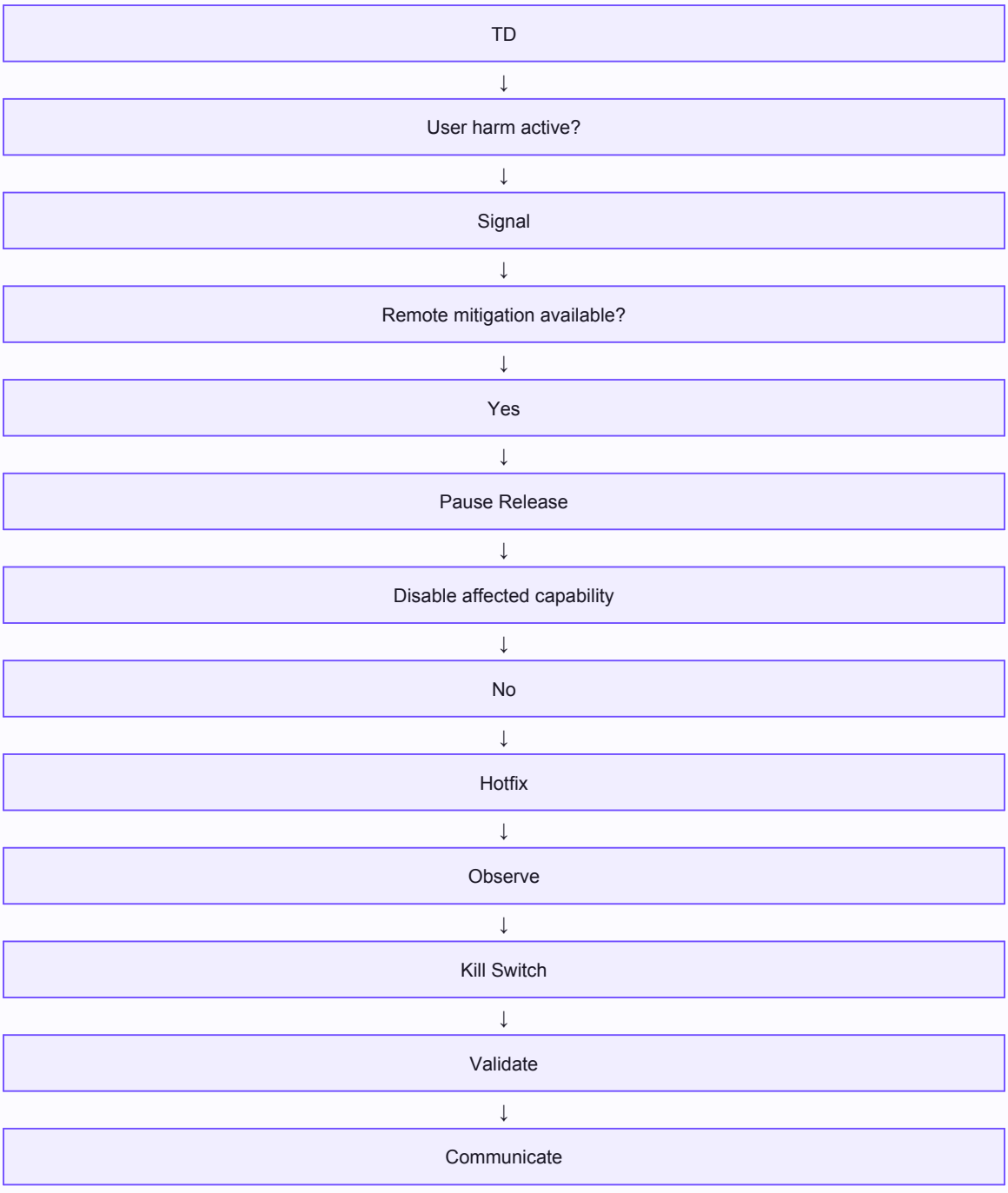
Release Candidate Flow

Public quality workflow diagram



Rollback Flow

Public quality workflow diagram

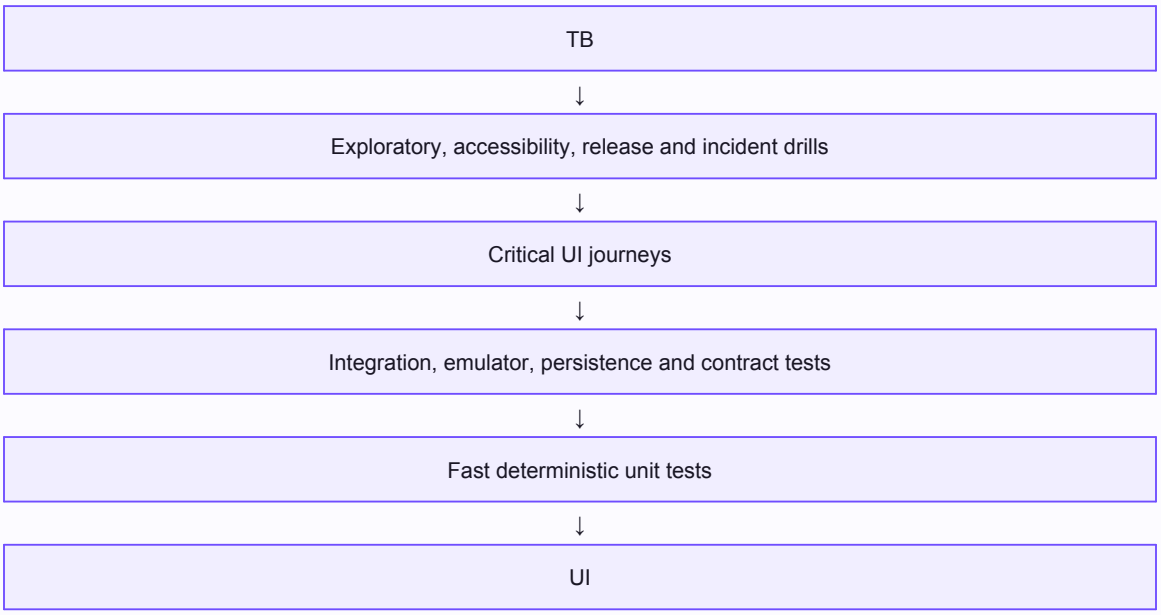


Support Triage

Public quality workflow diagram

Test Pyramid

Public quality workflow diagram



App Store Readiness Checklist

- □ TestFlight exit criteria are met by the same release candidate.
- □ No open critical defect or unaccepted high-risk exception exists.
- □ Privacy policy and App Privacy labels match actual data flows.
- □ TMDB attribution, content rights and third-party notices are correct.
- □ Screenshots, metadata, age rating, export compliance and review notes are verified.
- □ Account deletion is discoverable and demonstrably complete.
- □ Accessibility gate passes.
- □ Monitoring, billing alerts, incident ownership and rollback/hotfix path are active.
- □ Archive, dSYMs, release evidence and support URL are retained.
- □ Release mode and post-release validation window are approved.

Release Candidate Checklist

- □ One designated candidate build and immutable commit/tag.
- □ Production configuration, bundle ID, entitlements and signing verified.
- □ Clean archive and installation on physical device.
- □ Automated checks pass without ignored failures.
- □ Manual smoke, accessibility, privacy, offline/sync and performance evidence attached.
- □ Known limitations have owners, workarounds and dispositions.
- □ Release notes and changelog reconcile with scope.
- □ Rollback, kill switch or hotfix strategy documented.

Smoke Test Suite

1. Install/upgrade and launch.
2. Sign in, sign out and relaunch.
3. Complete profile/onboarding.
4. Create a past watch and a full journal entry.
5. Edit, view and delete an entry.
6. Search/filter timeline.
7. Generate and refresh QuickPick.
8. Add/remove watchlist item.
9. Create/join/leave Circle and verify privacy boundaries.
10. Exercise offline create, terminate, reconnect and confirm one synchronized result.
11. Verify settings, privacy links and account deletion entry point.
12. Run VoiceOver through one complete core journey.

TestFlight Readiness Checklist

- □ Scope and known limitations are explicit.
- □ Candidate builds cleanly from a tagged commit.
- □ Unit, integration and UI smoke suites pass.
- □ Core manual smoke passes on minimum and current iOS.
- □ Authentication and account recovery paths are verified.
- □ Offline create/update/delete survives termination and reconnection.
- □ Privacy and Firestore authorization matrix passes.
- □ VoiceOver and Dynamic Type core journeys pass.
- □ Crash symbols upload and a controlled crash is symbolicated.
- □ Beta review information, test account and contact data are current.
- □ Tester cohorts, instructions, feedback path and exit criteria are defined.

Accessibility Matrix

Public validation matrix

Dimension	Scope	Method	Gate
VoiceOver	Launch, onboarding, add/edit, timeline, QuickPick, settings, deletion	Manual + targeted UI assertions	Release blocking
Dynamic Type	All core screens through accessibility sizes	Manual matrix / snapshots where stable	Release blocking
Reduce Motion	Transitions and animated feedback	Manual	Release blocking for task completion
Contrast	Text, icons, controls, states	Audit + manual	Release blocking
Touch targets	Interactive controls	Audit + manual	Release blocking
Focus and announcements	Forms, errors, modals	Manual VoiceOver	Release blocking

Device Matrix

Public validation matrix

Device class	OS	Gate	Scope
Primary current iPhone	Latest supported iOS	Required	Full smoke, core journeys, accessibility
Oldest supported iPhone	Minimum supported iOS	Required	Install, launch, core journeys, performance
Small-screen iPhone	Minimum and latest	Required	Layout, keyboard, Dynamic Type
Large-screen iPhone	Latest	Required	Layout and large dataset
iPad, if supported	Minimum and latest	Conditional	Orientation, navigation, layout
Physical device	Release OS	Required	Sign in with Apple, notifications, performance, battery

Offline Sync Matrix

Public validation matrix

Scenario	Method	Expected	Risk
Offline create	Create entry, terminate, relaunch, reconnect	No loss; one remote record; synced state	Critical
Offline update	Edit synced entry offline and reconnect	Latest valid intent preserved under conflict policy	Critical
Offline delete	Delete offline, terminate, reconnect	No resurrection; remote soft delete consistent	Critical
Queue retry	Transient and permanent errors	Backoff; bounded retries; actionable failed state	Critical
Deduplication	Repeated enqueue/retry	Single logical action/result	Critical
Multi-device	Concurrent edits/share/membership	Policy applied; no unauthorized stale access	Critical

Privacy Security Matrix

Public validation matrix

Control	Verification	Method	Gate
Entry ownership	Cross-user reads/writes denied	Firestore emulator + integration	Block
Circle membership	Non-member and removed-member access denied	Rules + multi-device	Block
Private sharing	Only intended recipients can access	Rules + UI/integration	Block
Account deletion	Local and server data removed per policy	End-to-end audit	Block
Logs	No tokens, invite codes, content or PII	Static/manual log review	Block
Analytics	Events minimized and labels accurate	Schema and App Store label review	Block

Risk Matrix

Public validation matrix

ID	Risk	Severity	Likelihood	Disposition	Required evidence
R-001	Silent data loss during offline create/update/delete	Critical	Possible	Block release	Queue, persistence, forced termination, retry, reconciliation, multi-device
R-002	Unauthorized Circle or shared-entry access	Critical	Possible	Block release	Firestore emulator rules tests plus negative authorization matrix
R-003	Account deletion leaves remote or local personal data	Critical	Possible	Block release	End-to-end deletion verification and retained-data inventory
R-004	Authentication routing traps user or exposes another session	Critical	Possible	Block release	State-machine tests across launch, revoke, sign-out, profile failure
R-005	Duplicate or conflicting entries after reconnection	High	Likely	Block for critical flows	Conflict, dedupe and idempotency integration suite
R-006	Accessibility prevents task completion	High	Possible	Block release	VoiceOver, Dynamic Type, focus, labels, contrast and Reduce Motion regression
R-007	TMDB attribution/privacy metadata is incorrect	High	Possible	Block submission	Manual and automated compliance verification
R-008	Crash spike after release	High	Possible	Pause/rollback	Crashlytics symbol validation, phased release and thresholds
R-009	Firebase billing or function usage spike	High	Possible	Operational gate	Budgets, alerts, dashboards, rate limits and runbook
R-010	Large library causes unusable performance	Medium	Possible	Conditional	Synthetic large-dataset benchmarks and launch/timeline budgets